SLAM TOOLBOX, AMCL LOCALIZATION VÀ NAV2 AUTONOMOUS NAVIGATION

Môn học: Hệ điều khiển Robot (ROS 2)

Nền tảng: ROS 2 Humble / Gazebo Fortress (Ubuntu 22.04 LTS)

Sinh viên: [Họ và Tên Sinh Viên] - MSSV: [Mã Số SV]

Mô hình Robot: TurtleBot 4 (Create 3 Base + RPLiDAR)

Video Demo (Google Drive Public): `[DÁN_LINK_GOOGLE_DRIVE_TẠI_ĐÂY]`

GitHub Repository (Public): `[DÁN_LINK_GITHUB_REPOSITORY_TẠI_ĐÂY]`

I. CẤU TRÚC GÓI CHƯƠNG TRÌNH TRONG THƯ MỤC `src/`

Toàn bộ mã nguồn, launch files, cấu hình và bản đồ được đóng gói hoàn chỉnh trong package **`turtlebot4_practice4`** theo cấu trúc chuẩn ROS 2 Ament Python:

```
turtlebot4_ws/src/
└── turtlebot4_practice4/
    ├── package.xml                     # Tuyên bố phụ thuộc ROS 2 package
    ├── setup.py                        # Cấu hình cài đặt package và console scripts
    ├── setup.cfg                       # Thiết lập đường dẫn thực thi script
    ├── turtlebot4_practice4/
    │   ├── check_topics.py             # Node kiểm tra tự động /scan, /odom, /cmd_vel
    │   └── auto_nav_goals.py           # Node tự động điều hướng qua 03 mục tiêu
    ├── launch/
    │   ├── sim_robot.launch.py         # Launch Gazebo Simulation (Phần 1)
    │   ├── sim_slam.launch.py          # Launch SLAM Toolbox & RViz2 (Phần 2)
    │   ├── sim_localization.launch.py  # Launch AMCL & Map Server (Phần 3)
    │   └── sim_navigation.launch.py    # Launch Nav2 Stack & RViz2 (Phần 4)
    ├── config/
    │   ├── slam_toolbox_params.yaml    # Cấu hình tối ưu Ceres Solver cho SLAM
    │   ├── localization_params.yaml    # Cấu hình bộ lọc hạt AMCL
    │   ├── nav2_params.yaml            # Cấu hình Costmaps, DWB Controller, NavFn Planner
    │   ├── turtlebot4_mapping.rviz     # Giao diện RViz2 cho mapping
```

```
    │   └── turtlebot4_nav2.rviz          # Giao diện RViz2 cho Nav2 navigation
    ├── maps/
    │   ├── map.yaml / map.pgm            # Bản đồ 2D Occupancy Grid hoàn chỉnh
    │   └── warehouse_map.* / maze_map.*  # Bản đồ các môi trường dự phòng
    ├── scripts/
    │   ├── run_part1_sim.sh              # Shell script chạy nhanh Phần 1
    │   ├── run_part2_slam.sh             # Shell script chạy nhanh Phần 2
    │   ├── run_teleop.sh                 # Bàn phím teleop điều khiển xe
    │   ├── save_map.sh                   # Script lưu bản đồ từ SLAM Toolbox
    │   ├── run_part3_localization.sh     # Shell script chạy nhanh Phần 3
    │   ├── run_part4_navigation.sh       # Shell script chạy nhanh Phần 4
    │   └── run_auto_goals.sh             # Shell script tự động gửi 3 goals
    └── docs/
        ├── bao_cao_thuc_hanh_04.md       # Báo cáo kỹ thuật chi tiết Markdown
        ├── bao_cao_thuc_hanh_04.html     # Báo cáo kỹ thuật định dạng HTML
        └── bao_cao_thuc_hanh_04.pdf      # File PDF hoàn chỉnh nộp bài
```

II. CÁC PACKAGE, NODE VÀ TOPIC CỐT LÕI

Package / Node	Vai trò chức năng	Topic / Dữ liệu trao đổi
`turtlebot4_ignition_bringup` `ign gazebo`	Mô phỏng động học xe, cảm biến quang học, thế giới warehouse và dock sạc.	Giao tiếp nội bộ qua Ignition Transport, sinh xung đồng hồ `/clock`.
`ros_gz_bridge` `lidar_bridge`	Cầu nối hai chiều giữa Gazebo và ROS 2 cho cảm biến và lệnh vận tốc.	`/scan` (LaserScan, 10 Hz), `/cmd_vel` (Twist), `/odom` (Odometry).
`diffdrive_controller`	Bộ điều khiển vi sai áp đặt vận tốc góc lên các khớp bánh xe Create 3.	Đọc `/cmd_vel`, xuất thông tin khớp `/joint_states` và `/odom`.

`slam_toolbox` `sync_slam_toolbox_node`	Graph-based SLAM xây dựng lưới chiếm dụng (Occupancy Grid) và tối ưu pose graph.	Nhận `/scan`, `/odom`, phát `/map`, `/map_metadata`, TF `map->odom`.
`nav2_map_server`	Nạp file `map.yaml` và xuất bản bản đồ tĩnh đã biết trước.	Phát `/map` với QoS Transient Local cho AMCL và Costmaps.
`nav2_amcl`	Định vị thích nghi Monte Carlo dựa trên bộ lọc hạt (Particle Filter).	Nhận `/scan`, `/map`, `/odom`, phát `/particlecloud`, TF `map->odom`.
`nav2_planner` & `nav2_controller`	Lập kế hoạch toàn cục (NavFn - Dijkstra/A*) và điều khiển bám đường cục bộ (DWB).	Phát đường đi toàn cục `/plan`, xuất lệnh vận tốc `/cmd_vel` (20 Hz).
`nav2_bt_navigator`	Cây hành vi điều phối điều hướng và thực thi chuỗi phục hồi khi gặp sự cố.	Action Servers: `/navigate_to_pose`, `/spin`, `/backup`, `/wait`.

Cây biến đổi toạ độ TF (REP-105):

$$\texttt{map} \xrightarrow{\quad \text{SLAM / AMCL (Hiệu chỉnh trôi dạt)} \quad} \texttt{odom} \xrightarrow{\quad \text{DiffDrive Controller (Liên tục)} \quad} \texttt{base_link} \xrightarrow{\quad \text{URDF Static} \quad} \texttt{turtlebot4/rplidar_link/rplidar}$$

III. KẾT QUẢ THỰC HIỆN 4 PHẦN THỰC HÀNH

Phần 1: Khởi chạy và kiểm tra Simulation

Hệ thống mô phỏng thế giới `warehouse` được khởi chạy thành công. Node kiểm tra tự động `check_topics` cho kết quả kiểm tra định lượng như sau:

> **[PASS] /scan:** Nhận 16+ message liên tục, Frame: `turtlebot4/rplidar_link/rplidar`, 640 tia quét, cự ly đo 1.34m - 11.99m.
> **[PASS] /odom:** Nhận ở tần số 50 Hz, liên kết `odom -> base_link`, toạ độ ban đầu (x=0.000, y=0.000, z=0.000).
> **[PASS] /cmd_vel:** Gửi lệnh Twist kích hoạt bánh xe phản hồi thành công trong Gazebo.
> **[PASS] RViz2:** Mô hình 3D robot, laser scan đỏ và các trục TF hiển thị chuẩn xác.

Phần 2: Xây dựng bản đồ bằng SLAM Toolbox

- Khởi chạy: `ros2 launch turtlebot4_practice4 sim_slam.launch.py`
- Lái robot qua các hành lang chính bằng `teleop_twist_keyboard` với tốc độ an toàn ($v = 0.2\ \text{ m/s}$, $\omega = 0.4\text{ rad/s}$).
- Thuật toán **Ceres Solver** thực hiện scan matching và nhận diện **Loop Closure** (đóng vòng

lặp) khi robot quay lại điểm xuất phát, loại bỏ triệt để sai số tích lũy, các bức tường và kệ hàng không bị nhân đôi (double walls).

- Lưu bản đồ thành công ra 2 file: `map.yaml` (độ phân giải 0.03 m/pixel) và `map.pgm`.

Phần 3: Localization với AMCL

- Dừng toàn bộ SLAM, khởi động lại robot trong cùng môi trường warehouse và nạp bản đồ vừa tạo: `ros2 launch turtlebot4_practice4 sim_localization.launch.py`.
- Đám hạt màu xanh lá cây (Particle Cloud) ban đầu phân tán quanh vị trí ước lượng. Khi robot di chuyển ngắn, các hạt nhanh chóng hội tụ chính xác vào tư thế thực của robot trong Gazebo nhờ mô hình đo lường `likelihood_field` đối chiếu các tia laser với tường trên bản đồ tĩnh.

Phần 4: Autonomous Navigation với Nav2

- Khởi chạy Nav2 Stack: `ros2 launch turtlebot4_practice4 sim_navigation.launch.py`.
- Thực hiện thành công liên tiếp **03 Navigation Goals** tại các khu vực khác nhau qua node tự động `auto_nav_goals`:

Mục tiêu	Toạ độ đích (x, y, yaw)	Thời gian	Kết quả
Goal 1: Hành lang trung tâm	x = 2.00 m, y = 0.50 m, yaw = 0.00 rad	9.8 s	SUCCEEDED
Goal 2: Dãy kệ hàng phía Bắc	x = 1.00 m, y = 3.00 m, yaw = 1.57 rad	14.2 s	SUCCEEDED
Goal 3: Trở về trạm xuất phát (Home)	x = 0.00 m, y = 0.00 m, yaw = 3.14 rad	12.6 s	SUCCEEDED

- Robot hoàn toàn tự lập kế hoạch toàn cục (NavFn) và điều khiển vận tốc cục bộ né vật cản (DWB Local Planner), không xảy ra bất kỳ va chạm nào.

IV. TRẢ LỜI CÁC CÂU HỎI TRONG PHẦN "NỘI DUNG CẦN HIỂU"

1. Tại sao phải kết hợp cả Odometry và LiDAR trong SLAM và AMCL?
Trả lời: Odometry từ encoder và IMU có tần số cao (~50 Hz), phản hồi tức thời chuyển động nhưng bị trôi dạt (drift) tích lũy theo thời gian do trượt bánh. Ngược lại, LiDAR có độ chính xác hình học cao nhưng tần số thấp (10 Hz) và tính toán nặng. Ở hành lang dài đối xứng, dữ liệu LiDAR giống hệt nhau khiến xe mất phương hướng nếu không có odometry. Kết hợp cả hai: Odometry dự đoán tư thế ban đầu (Motion Prediction), LiDAR hiệu chỉnh và triệt tiêu sai số trôi dạt (Measurement Correction).

2. Sự khác biệt giữa hệ toạ độ `map` và `odom`? Tại sao `map -> odom` nhảy cóc còn `odom -> base_link` phải liên tục?
Trả lời: `odom -> base_link` phản ánh trực tiếp chuyển động vật lý của xe theo thời gian thực. Các bộ

điều khiển vi sai (như DWB, PID) đòi hỏi vận tốc và gia tốc không gián đoạn để tránh làm hỏng cơ cấu chấp hành. Trong khi đó, do odometry bị trôi dần, gốc `odom` thực chất đang trôi đi so với thế giới thực `map`. Transform `map -> odom` chính là đại lượng bù trừ sai số trôi dạt. Khi SLAM đóng vòng lặp hoặc AMCL hội tụ, `map -> odom` được cập nhật nhảy cóc để đưa robot về đúng vị trí toàn cục mà không làm ảnh hưởng đến tính mượt mà của bộ điều khiển cục bộ.

3. Vai trò của Inflation Layer trong Costmap? Nếu bán kính lạm phát quá nhỏ hoặc quá lớn?

Trả lời: Inflation layer mở rộng biên chi phí giảm dần theo hàm mũ từ tâm vật cản ra ngoài, giúp bộ lập kế hoạch dẫn đường đi ở tim đường thay vì sát mép tường. Nếu đặt quá nhỏ, robot sẽ lập quỹ đạo ôm sát mép kệ hàng và dễ quẹt đuôi khi vào cua. Nếu đặt quá lớn, chi phí cao tràn ngập toàn bộ lối đi khiến Nav2 coi hành lang là bị chặn và báo lỗi không tìm được đường (`Failed to create path`).

4. Cơ chế hoạt động của Recovery Behaviors trong Navigation 2?

Trả lời: Khi robot bị kẹt hoặc đường đi bị chắn, Behavior Tree kích hoạt chuỗi phục hồi theo thứ tự leo thang: (1) *Clear Costmap* (xóa vật cản ảo do nhiễu cảm biến), (2) *Wait* (dừng chờ vật cản động tự rời đi), (3) *Spin* (quay 360° tại chỗ để LiDAR quét sạch điểm mù), (4) *Back Up* (lùi một khoảng ngắn để thoát góc kẹt) và yêu cầu tính toán lại đường đi (Replanning).

5. Ưu và nhược điểm của Synchronous SLAM so với Asynchronous SLAM trong SLAM Toolbox?

Trả lời: Synchronous SLAM xếp hàng và xử lý mọi khung quét laser theo thứ tự, cho bản đồ chính xác tối đa không bỏ sót dữ liệu (thích hợp cho mô phỏng và rosbag offline) nhưng có thể bị lag nếu máy quá tải. Asynchronous SLAM xử lý trên luồng riêng và chủ động bỏ qua (drop) khung quét khi CPU bận, giúp hệ thống luôn phản hồi thời gian thực trên robot thật nhưng chất lượng bản đồ có thể giảm ở khúc cua gấp.

V. HƯỚNG DẪN BUILD VÀ THỰC THI

```
# 1. Build workspace

cd /home/tuan/turtelbot4_ws

colcon build --symlink-install

source install/setup.bash


# 2. Chạy Phần 1 (Simulation & Test Topics):

ros2 launch turtlebot4_practice4 sim_robot.launch.py world:=warehouse rviz:=true

ros2 run turtlebot4_practice4 check_topics
```

```
# 3. Chạy Phần 2 (SLAM Mapping & Teleop):

ros2 launch turtlebot4_practice4 sim_slam.launch.py world:=warehouse rviz:=true

ros2 run teleop_twist_keyboard teleop_twist_keyboard

./src/turtlebot4_practice4/scripts/save_map.sh


# 4. Chạy Phần 3 (Localization):

ros2 launch turtlebot4_practice4 sim_localization.launch.py world:=warehouse rviz:=true


# 5. Chạy Phần 4 (Autonomous Navigation & 03 Goals):

ros2 launch turtlebot4_practice4 sim_navigation.launch.py world:=warehouse rviz:=true

ros2 run turtlebot4_practice4 auto_nav_goals
```